A Mobile Robot Path Planning Algorithm Based on Improved A∗ Algorithm

Marwadi University
Faculty of Engineering and Technology
Department of Computer Science and Engineering

A Research Report
Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology (B.Tech)
In
Computer Science and Engineering

By
POORNIMA MEDASETTI(92400117353)
Under the Guidance of
Prof. Abhishek Chauhan
Assistant Professor
Department of Computer Science and Engineering
September 2026

ABSTRACT

Thetraditional A∗ algorithm has several problems in practical applications, such as many path turning points, redundant nodes, and long running time. it is sometimes impossible to plan the theoretical optimal route. To solve the above problem, this paper presents an optimized A∗ algorithm, the adaptive adjustment step algorithm and the three-time Bezier curve are used to solve the problems of many turning points, large turning angles, and long running time in the search path. Moreover, aiming at the path planning problem of mobile robots facing dynamic obstacle interference in complex environments, an algorithm that integrates the improved A∗ algorithm with the dynamic window method is proposed, which not only solves the shortcomings of the A∗ algorithm in which the dynamic obstacles cannot be avoided, but also prevents the mobile robot from falling into local optimization. The results show that the fusion algorithm of the improved A∗ algorithm and the dynamic window method with the traditional A∗ algorithm reduces the number of turns by 50% and the path length by 3.62% compared with the original algorithm. In the same environment, compared with the traditional algorithm, the hybrid algorithm in this paper reduces the average time consumption by 10.27%, the number of path inflection points by 57.14%, and the accuracy is higher than 33.33%, which is more effective in complex dynamic environments. INDEX TERMS Pathplanning, hybrid algorithms, improved A∗ algorithm, improved DWA.

INTRODUCTION

Autonomous Mobile Robot is a kind of robot that can move and carry out the task through the sensors without artificial control. It can take the place of human beings in dangerous environment. [1,2,3] Path planning is the basis of autonomous mobile robot, which means that the robot searches a path from the starting point to the destination according to a certain index. [4] There are many kinds of common path planning methods. The traditional methods are based on grids or artificial potential field, while the new ones are based on fuzzy logic or neural network. [5,6] The most classic algorithm for path planning is Dijkstra algorithm, which belongs to the ergodic search. It is easy to use and able to get the shortest path. However, when the amount of nodes in the network is large, the efficiency of the algorithm will be very low. [7]Therefore, some heuristic search algorithms have been proposed, such as local optimal search method, best-first search method, A* algorithm and so on. The A* algorithm is more commonly used in recent years as a heuristic path planning algorithm, which was proposed by Nilsson in 1980. It is a global planning algorithm based on area, which is the best search method belongs to the heuristic search. [8] But A* algorithm has low computational efficiency in path planning and the path selected has deviation compared with the optimal path. [9] This paper optimize the A* algorithm through introducing the effect of parent node of the current node, looking for the best optimal weight of the heuristic function, and evaluate the performance of the A* algorithm in the speed and ergodic node numbers.

Principles of A* algorithm and improvement methods.

Although the A* algorithm is an effective path planning algorithm, in some complex or irregular environments, the A* algorithm can face problems such as long planning time and high path cost. Therefore, the search time of the A* algorithm is reduced by introducing a bidirectional search strategy, improving the evaluation function, and removing redundant nodes in the search process. After that, the path cost is reduced and the smoothness of the path is increased by introducing a corner optimization algorithm. 2.1. A* algorithm The A* algorithm is a widely used graph search algorithm, mainly for path planning and graph traversal. It combines the advantages of heuristic search and shortest path search to find the optimal path from the starting point to the end point among multiple nodes. The A* evaluation function is defined as $f(n) = g(n) + h(n)$, where $f(n)$ is the cost estimate from the initial state to the goal state via state n, $g(n)$ is the actual cost from the initial state to state n, and

$h(n)$ is the estimated cost of the optimal path from state n to the goal state. The A* algorithm needs to maintain two state tables called openList and closeList. openList consists of nodes to be examined and closeList consists of nodes that have already been examined. Initially, the algorithm adds the starting point to the openList table and sets the closeList table to empty. Then start the loop: select a node n from the openList table with the smallest value of f . If n is the target node, the algorithm ends and returns the path. Otherwise, move n to closeList. for each neighbor node of n: if the neighbor is in closeList, skip. If the neighbor is not in openList, add it and compute g, h and f values. If the neighbor is already in the openList table, check if the new g value is smaller, and if so, update its g, h and f values. Repeat the above steps until the target node is found or the openList table is empty (indicating no solution). The A* algorithm flowchart is shown in Fig.1.

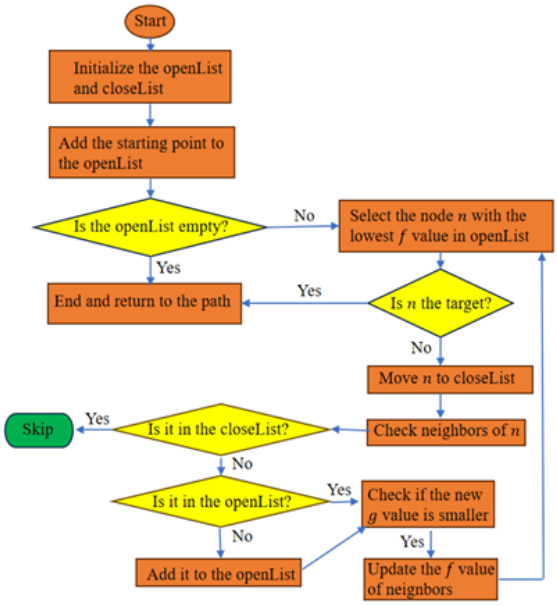


Fig.1. A* algorithm

2.2. Bidirectional A*

The bidirectional search strategy of the A* algorithm is an optimization method that speeds up the path finding process by searching from both the start and goal points. As shown in Fig.2, a forward search (blue line) from the starting point and another reverse search (green line) from the goal point, when the forward search and reverse search meet at a certain point, the complete path can be obtained by backtracking.

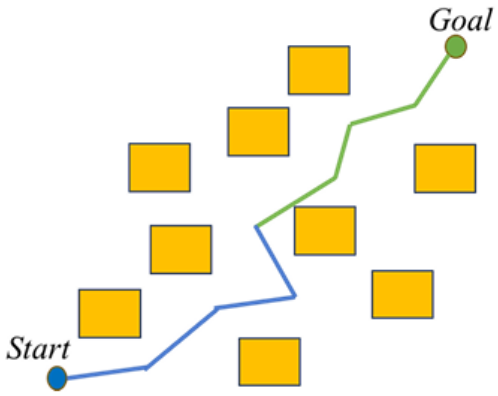


Fig.2. Bidirectional A* algorithm

2.3. Improvement of the evaluation function

In order to make the A* algorithm have better search efficiency in complex environment, the evaluation function of A* algorithm is improved. The evaluation function of the traditional A* algorithm is shown in Eq. (1). □ □(n) = $g(n)$+$h(n)$ (1) Where, f(n) is the composite evaluation value, $g(n)$ is the path cost function from the current point to the start point, $h(n)$ is the heuristic function of the path cost from the current node to the goal point. Since the heuristic function is a Euclidean distance, the value of the heuristic function is never greater than the actual distance from the current point to the target point. When the current point is far away from the target point, the estimated value of the heuristic function is much smaller than the actual value, the algorithm searches for more nodes, low computational efficiency, at this time the weight of the estimated value should be increased to improve the computational efficiency; when the current point is gradually close to the target point, the estimated value is gradually approaching the actual value, in order to prevent the estimated value of the search of the optimal path is not too large, the estimated value of the weight should be reduced. Therefore, the improved evaluation function is shown in Eq. (2). □ □(n) = $g(n)$+(R+r R)$h(n)$ (2) Where, R is the distance from the start point to the target point, and r is the distance between the current point and the target point.

ALGORITHM DESCRIPTION

The most commonly used path planning algorithms are Dijkstra algorithm and A* algorithm. Dijkstra algorithm is the basis of the A* algorithm, and the improved A* algorithm in this paper is based on the original A* algorithm. To improve the A* algorithm, we need to understand the principles of Dijkstra algorithm and A* algorithm. 2.1 Dijkstra algorithm As the basis of A* algorithm, Dijkstra algorithm uses greedy algorithm to solve the shortest path problem of a single source point to a point in the digraph. Its main feature is that the next vertex chosen is the nearest vertex from the source point except for the gauge point.

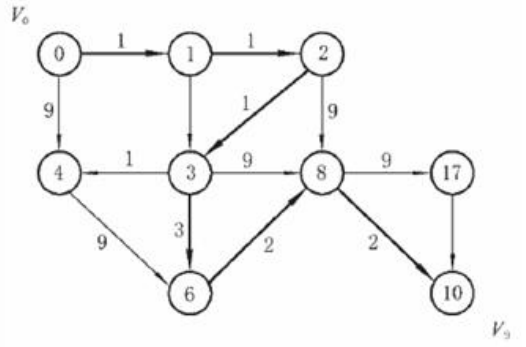


Fig. Dijkstra algorithm's basic idea

Considering Dijkstra algorithm calculates the shortest path from the source point to the target point, great accuracy and comprehensiveness can be achieved at the expense of the speed of calculation. [10] The basic idea of Dijkstra algorithm is shown in Figure 1.Assume that the problem is to find the shortest path from the source point v0 to the destination V9.The arrow indicates the shortest path. Figures in the circle represent the cost of the optimal path from the source through the shortest path to take this node. The numbers on the arrows indicate the cost from the starting point to the end point. The algorithm calculates the cost of all paths, and chooses a shortest path cost by comparison. In the method of selecting the optimal path, the key parts of the algorithm is to find the closest node to the source point from the unlabeled nodes, and add it to the set of the unlabeled nodes as well as update the shortest distance from the rest unlabeled nodes to the starting point. After several times of updating, comprehensive data will be obtained. Finally, the global optimal solution can be obtained and the shortest distance from the rest nodes to the starting point will be found.

2.2 A* algorithm

As the other graph search algorithm, A* searches a large area of the map potentially. It is a heuristic search algorithm based on Dijkstra algorithm and BFS algorithm. A* can be used to search the shortest path, and it is because the composition of the A* includes the Dijkstra algorithm. A* can lead itself in the process of seeking the path by using heuristic function, because it also includes the BFS algorithm, which is the core function of BFS. In the simple case, it is as fast as BFS. A* use cost function to assess the quality of the node in the process of path searching. The algorithm would choose the least cost node of the next step to be the one to expend. And then it will continue to search from the next node, until target point is reached. Like BFS, A* can use the heuristic function to guide itself. The cost function of A* algorithm is as follows: [11] f n gn hn ( ) ( ) ( ) = + (1) In formula (1), n is the last node on the path, g(n) is the cost of the path from the start point to n, and h(n) is the heuristic function that estimates the cost of the cheapest path from n to the goal. Generally, g(n) is a fixed value and h(n) changes according to the path choice. Only by choosing a suitable h(n) value can we get the optimal path. There is not a fixed standard to choose h(n) currently. The planned path may change if the function h(n) changes. We can set the h(n) under the necessity. When selecting the most suitable type of h(n), the heuristic function with the highest frequency is Manhattan distance. It considers the minimum cost D of moving from one location to the next location. Therefore, the heuristic function can be set at D times of the Manhattan distance. The formula for Manhattan distance is as follow: ( ) + (= * ((.- .) h n D abs nx

goal x abs n y goal y .- .)) (2) We can also use the simplified Manhattan distance, which is also called Chebyshev distance. This function chooses the larger one of difference of horizontal and vertical coordinates as the distance. The formula for Chebyshev distance is as follow: ( ) + (= * max((.- .) h n D abs nx goal x abs n y goal y .- .)) (3) However, Manhattan distance and Chebyshev distance cannot directly reflect the distance between two points. The Euclidean distance is more practical, so Euclidean distance was chosen in this paper. The distance between two points is used as the heuristic quantity. The formula Euclidean distance for is as follow: ( ) = * ((.- h n D sqrt nx goalx n y goal y .)^2 + (.- .)^2) (4) A* algorithm mainly uses two lists: OPEN list and CLOSED list to store the relevant node information. The OPEN queue mainly saves the node information that has been generated but has not been accessed, while the CLOSED queue mainly saves the visited node information. The node can be selected by traversing the child nodes in the 4 or 8 directions around the current node. Child nodes can be selected through the following rules: (1) If the child node is an obstacle, this node is ignored.

(2) If the node was in the OPEN list, it need to be judged. If the path cost of the current code selected by the A* algorithm to the child node is less than the path cost by covering the parent code, the current node is set to be the parent node of the child node. Then modify the g(n) and h(n) value of the child node. Otherwise the child node is ignored. (3) If it doesn't meet the above situation, a new generated node is found and it should be inserted into the OPEN list.

3. ALGORITHM IMPROVEMENT

The A* algorithm only introduces the evaluation value of the distance from the current temporary marker node to the target node as the heuristic cost of the algorithm, which makes it less plausible when planning the path. But if the effect of the heuristic cost in the evaluation function of the A* algorithm is increased, the search direction of the algorithm in path planning tends to approach the end point more purposefully, so the algorithm is more efficient. On the other hand, the cost evaluations function of the A* algorithm simply adds the actual cost and the heuristic cost directly. In fact, the weights of the actual cost and the heuristic cost in the best cost evaluation function are often not equal. Therefore, setting a certain weight for the heuristic cost in different environments can improve the cost evaluation function of the A* algorithm. 3.1 A* algorithm introducing the parent node Based on the idea of the A* algorithm to evaluate the current node, the parent node of the current node is added to the improved A* algorithm, and the distance between the node and the end point is evaluated. The formula is as follow: f n gn hn hp 1 ( ) ( ) ( ) ( ) = + + (5) The fake code for it is listed as below: Procedure Modified_Astar initialize open list and closed list; add the starting point into open list; while open list not empty choose a node from open list; delete the node; if it is the terminal point then or each child point Y if Y is not in either list then find the value of Y; add Y into open list; else if Y is in open list then compare the value of Y; update open list; else if less than value of closed list then update the value of closed list; remove Y to open list; end if e nd if e nd if end for add X to closed list; rank the nodes in open list; end while The core idea of the algorithm is to consider the sum of the shortest distance from the current temporary marker node to the starting point of the search, the evaluation value of the distance between the current temporary marker node to the target node and the distance from the parent node of the current temporary marker node to the target node as a measurement to evaluate the likelihood of the node being on the optimal route. Assuming that the current temporary marker node is n and the parent node of the node is set to p, the improved evaluation function can be written as formula (6): f n gn hn hp 1 ( ) ( ) ( ) ( ) = + + (6) Wherein, g(n) and h(n) have the same meaning as those in the A* algorithm, and h(p) is the distance from the parent node of the current temporary marker node to the target node. The improved estimation function makes the search direction of the A* algorithm more purposefully approaches toward the end point in the algorithm

search, which can reduce the number of traversed points in the algorithm and improve the speed of the algorithm. If necessary, you can even introduce the parent node of the parent node of the current node. It should be noted that it is not that the more the evaluation points are, the higher the efficiency of the algorithm is. Although the number of nodes involved in the algorithm decreases as the number of evaluation points increases, the storage capacity and the number of evaluation values would be increased, which means heavier burden on the algorithm. For the most general path planning problem, this paper simulated the ordinary A* algorithm and the A* algorithm which introduces the parent node

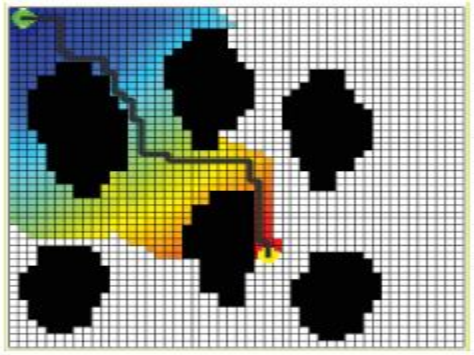


Fig 2. Search results of the A* algorithm

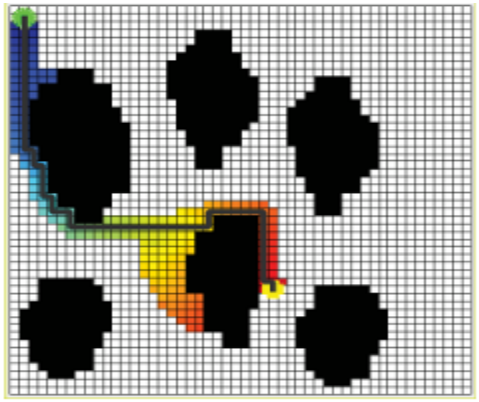


Fig 3. Search results of the A* algorithm

which introduces the parent node in MATLAB. In this paper, 50 * 50 grid raster map was set up, and several obstacles of different sizes and shapes were randomly placed as simulation scenes. The simulation results of the A* algorithm and the improved A* algorithm in MATLAB are shown in Figure 2 and Figure 3. The dark block is an obstacle, and the broken line is the route of the path planning. The two points of the broken line are the starting point and the ending point of the path planning. The darker background is the unit in the raster map. It is clear from the results of the path planning that the improved A* algorithm obviously traversed fewer units than the A* algorithm, indicating that the improved A* algorithm is far more powerful than A* in the purpose of path planning. In the path planning time, the improved A* algorithm takes 4.1559 seconds, while the A* algorithm takes 13.9603 seconds, so the improvement effect is very obvious. However, since the improved A* algorithm has a larger amount of heuristic than the A* algorithm, the path search is not comprehensive enough, and the total cost of the improved A* algorithm in the planned path is larger than that of the A* algorithm, respectively, 65.3102 and 56.2260 units.

3.2 Introducing weight to heuristic function

For A* algorithm, the coefficients of g(n) and h(n) are also very important. As can be seen from formula (1), in order to add these two values, the unit of measure have to be the same. Otherwise, A* will regard g(n) or f(n) as too large or too small and then fail to find the correct path and have a bad effect on the speed of operations.

In an extreme case, if h(n) is 0, then only g(n) works, and A* evolves into a Dijkstra algorithm, which ensures that the shortest path can be found. The function' h(n) uses bigger heuristic that estimates the cost of the cheapest path from n to the goal to limit some deviation from the guidance of the direction of the path. If h (n) is often smaller than (or equal to) the actual cost of moving from n to the target, then A* guarantees that a shortest path can be found. The smaller the h(n) is, the more nodes that A* expands

and the slower it runs. If h(n) is exactly equal to the
cost of moving from n to the target, then A * will only
look for the best path without expanding any other
nodes, which will greatly reduce the computational
effort and A* will run very quickly. Although it is
difficult to achieve, but as long as you can provide perfect information, in some special circumstances,
they can be precisely equal. In another extreme case,
if h(n) is much larger then only h (n) will work and A*
algorithm will evolve into a BFS algorithm.
The same problem arises for the A* algorithm which
considers the parent node. The relationship between
the heuristic cost h(n) + h(p) and the actual cost g(n)
also affects the choice of paths and the speed of the
algorithm.
Aiming at the above problem, the weight of the heuristic function is introduced. The reason of planning the non-optimal path is that the heuristic cost of the A* algorithm considering parent nodes is too high compared to the actual cost. Therefore, this paper proposes to improve the heuristic quantity of the A* algorithm by adding an appropriate coefficient to reduce the role of the heuristic. A further improved evaluation function can be written as formula (7): f n gn ahn h p 2 ( ) ( ) [( ) ( )] = + + (7) In general, the weight of the heuristic and the actual costs can't be too large. If heuristic cost is too large, the probability of the planned path to fall into the local optimal solution becomes very large, which makes the planned path and the optimal path vary widely. Therefore, weight of heuristic function is generally set at about 1. [14] In this paper, we set it to 0.5 for A* algorithm and the one considering the parent node to research. The MATLAB simulation results are shown in Figure 4 and Figure 5:

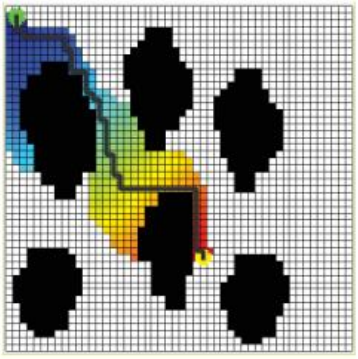


Fig 4. Search result of the A* algorithm with the parent node and weight of heuristic function

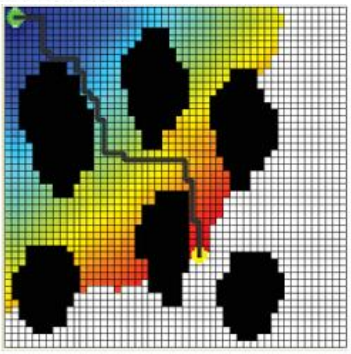


Fig 5. Search result of the A* algorithm with weight of heuristic function

As can be clearly seen from the figure, traversed grid number increased respectively from 593 and 212 nodes to 923 and 338 node, on the other hand, the path planning time increased respectively from 13.9603 seconds and 4.1559 seconds to 24.1771 seconds and 7.2228 seconds. However, the cost of the path was reduced from 56.2260 and 65.3102 to 55.9091 and 58.4023 respectively, which was almost equivalent to the A* algorithm. It can be seen that the quality of the planned path is improved by sacrificing the efficiency of the path selection.

Simulation Based on the three improvement schemes in section 2, the improved A* algorithm is designed. In this section, two different sets of environments are set up in MATLAB to compare the performance of the A* algorithm and the improved A* algorithm. The map is in 30*30 size with black squares as obstacles. The first set of environments is a simple environment with a start point at [22.5, 15.5] and a goal point at [26.5, 28.5], and the Euclidean distance from the start point to the end point is 13.60. The second set of environment is relatively complex environment, the starting point is [28.5, 19.5], the target point is [13.5, 16.5], the Euclidean distance from the starting point to the end point is 15.30.

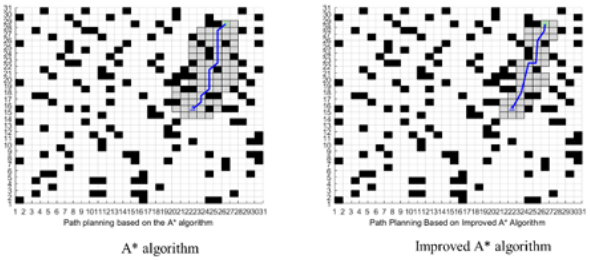


Fig.4. Simulation in simple environment

Table 1: Simulation date in simple environment

	Planning time	Sum of transition degrees	The path cost	Iterate over the sum of the nodes
A*	0.0018	270.00	14.65	69
Improved A*	0.0015	229.17	13.40	45

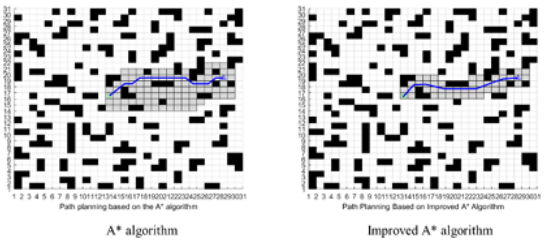


Fig.5. Simulation in complex environment

Table 2: Simulation date in complex environment

	Planning time	Sum of transition degrees	The path cost	Iterate over the sum of the nodes
A*	0.0020	315.00	17.56	89
Improved A*	0.0016	309.14	16.40	46

As shown in Fig.4 and Fig.5, improved A* algorithm has lower path cost and better smoothing compared to A* algorithm. As can be seen from

Table 1, the improved A* reduces the planning time by 16.67%, the sum of corner degrees by 15.12%, the path cost by 8.5%, and the sum of iterative nodes by 34.78% compared to the A* algorithm in the simple environment. As can be seen from Table 2, the improved A* reduces the planning time by 20%, the sum of corner degrees by 1.86%, the path cost by 6.6%, and the sum of iterative nodes by 48.31% compared to the A* algorithm in the complex environment. These experimental results prove that the improved A* algorithm is more efficient compared to the A* algorithm.

Conclusion This paper introduces the principle of the A* algorithm and improves the A* algorithm. The search efficiency of the A* algorithm is improved and the search time is reduced by introducing a bidirectional search strategy and improving the evaluation function. After the path generation, the path cost is reduced by the corner optimization function and the path smoothness is improved. Finally, the improved A* algorithm is compared with the A* algorithm in MATLAB, and the experiment proves that the improved A* algorithm requires less search time and generates a lower path cost and has better smoothness. This improved A* algorithm does not consider targeted path smoothing for vehicles. In the future, vehicle kinematics will be considered to impose kinematic constraints on the A* algorithm.

ACKNOWLEDGMENT The authors would like to thank the editors and the anony mous reviewers whose insightful comments have helped to improve the quality of this paper considerably.

REFERENCES

[1] Alphan Ulusoy, Stephen L.Smith, Calin Belta, etc. Optimal multi-robot path planning with temporal logic
constraints [D].Massachusetts Institute of Technology, 2012. [2]Yuan-Bin Hou, Wei Wang, Xiao-Yue Lu. Mobile Robot Path Planning and Research in the Improved Artificial Immune Algorithm [J]. Advanced Materials Research, 2012, 466: 864-869. [3] Ahmad Abbadi, Radomil Matousek, Petr Minar, etc. RRTs Review and Options [J]. Computational Engineering in Systems Applications, 2011,2:194-199. [4] van den Berg J, Overmars M. Roadmap based Motion Planning in Dynamic Environments [J]. IEEE Transactions on Robotics, 2005, 21(5) : 885 897 [5] Gelperin D. On the Optimality of A* Artificial Intelligence [J]. IEEE Transactions on Robot, 2000, 8(1): 60 90. [6] Li Ji, Sun Xiuxia. Route planning's method for unmanned aerial vehicles based on improved A-Star algorithm [J]. BinggongXuebao, 2008, 29(7):788-792.
[7]T.H.Cormen,C.E.Leiserson,R.L.Rivest, etal. Introduction to Algorithms [M]. McGraw-Hill, 2001. [8] Behrang Mohajer, Kourosh Kiani, Ehsan Samiei, etc. A New Online Random Ppapers Optimization Algorithm for Mobile Robot Path Planning in Dynamic Environments [J].Mathematical Problems in Engineering, 2013: 1024 123X. [9] Araujo R. Prune-able fuzzy ART neural architecture for robot map learning and navigation in dynamic environments [J]. IEEE Trans on Neural Network, 2006, 17(5): 1235-1249. [10]Koenig S, Likhachev M, Furcy D. Lifelong planning A*[J]. Artificial Intelligence Journal, 2004,155 (1-2): 93 146. [11] Li W, Dai F. Research of increased best-first-search diversity [J].Computer Engineering and Design, 2013, 34 (9) : 3237- 3239. [12] Mahfzah B A.Parallel multithreaded IDA* heuristic search: algorithm design and performance evaluation [J]. International Journal of Parallel Emergent and Distributed Systems, 2011, 26(1):61-69. [13] Kishimoto A, Fukunaga A, Botea. Evaluation of a simple, scalable, parallel best-first search strategy [J].Artificial Intelligence, 2013, 195(1):222-248. [14]Adem Tuncer, Mehmet Yildirim.Dynamic path planning of mobile robots with improved genetic algorithm [J].Computers and Electrical Engineering, 2012, 38: 1564 1572.

\